

Q. 1 Medium

Unanswered

Flag

Q029

Autonomous Last-Mile Delivery Trip Analytics

Domain: Autonomous Logistics / Quick Commerce

Difficulty: Medium

Marks: 20

Duration: 20–25 Minutes

Functions: 5

Tests: 6

Problem Statement

A quick-commerce platform uses delivery partners and autonomous vehicles for last-mile trips.

Each trip records:

- dispatch time
- promised time
- actual arrival time
- distance
- fare
- partner
- delay reason

The operations team needs trip-delay, speed, partner-performance, and delay-reason insights.
This follows the delay/time-analysis pattern used in Q216.

Dataset

data/delivery_trips.csv

Columns:

trip_id

partner

dispatch_ts

promised_ts

actual_ts

distance_km

fare_amount

delay_reason

status

Timestamp format:

yyyy-MM-dd HH:mm:ss

Task 1 Load Delivery Trips 4 Marks



Task 1 — Load Delivery Trips — 4 Marks

```
def load_delivery_trips(
```

```
    spark: SparkSession,
```

```
    path: str
```

```
) → DataFrame:
```

Requirements

Convert:

dispatch_ts

promised_ts

actual_ts

to TimestampType.

Cast:

distance_km → double

fare_amount → double

Task 2 — Compute Trip Metrics — 4 Marks

```
def compute_trip_metrics(
```

Task 2 — Compute Trip Metrics — 4 Marks

```
def compute_trip_metrics(  
    df: DataFrame  
    ) → DataFrame:
```

delay_hours

$(\text{actual_ts} - \text{promised_ts}) / 3600$

using `unix_timestamp()`.

Negative delays become:

0.0

trip_hours

$(\text{actual_ts} - \text{dispatch_ts}) / 3600$

Negative values become:

0.0

avg_speed_kmph

If:

trip_hours > 0

calculate:

distance_km / trip_hours

otherwise:

0.0

Task 3 — Filter Significant Delay Trips — 4 Marks

def filter_significant_delays(

df: DataFrame,

delay_threshold: float,

min_distance: float

) → DataFrame:

Keep:

Task 3 — Filter Significant Delay Trips — 4 Marks

```
def filter_significant_delays(
```

```
    df: DataFrame,
```

```
    delay_threshold: float,
```

```
    min_distance: float
```

```
) → DataFrame:
```

```
    Keep:
```

```
    delay_hours > delay_threshold
```

```
    and:
```

```
    distance_km >= min_distance
```

Task 4 — Partner Delay Summary — 4 Marks

```
def partner_delay_summary(
```

```
    df: DataFrame
```

```
) → DataFrame:
```

Task 4 — Partner Delay Summary — 4 Marks

```
def partner_delay_summary(  
    df: DataFrame  
    ) → DataFrame:
```

Keep:

```
status == "Completed"
```

Group by:

```
partner
```

Calculate:

```
SUM(delay_hours) → total_delay_hours
```

```
AVG(fare_amount) → avg_fare
```

```
COUNT(trip_id) → trip_count
```

Required output:

```
partner
```

Required output:

partner

total_delay_hours

avg_fare

trip_count

Task 5 — Highest Average Delay Reason — 4 Marks

```
def highest_average_delay_reason(
```

```
    df: DataFrame
```

```
) → Tuple[str, float]:
```

Ignore null:

delay_reason

delay_hours

Group by:

delay_reason



Q.1

Description

Q.2

delay_reason

Calculate:

AVG(delay_hours)

Alias:

avg_delay_hours

Sort:

avg_delay_hours DESC

delay_reason ASC

Return:

(delay_reason, avg_delay_hours)

Empty:

("", 0.0)

Return:

(delay_reason, avg_delay_hours)

Empty:

("", 0.0)

Tests

TC1 load_delivery_trips

TC2 compute_trip_metrics

TC3 filter_significant_delays

TC4 partner_delay_summary

TC5 highest_average_delay_reason

TC6 tie + empty behavior